

Rapport de Conception — 3 Games Hub

Auteur : Aaron Cabling
Contexte : Projet de développement web — Licence 3 MIAGE Date : 20 avril 2026

1. Présentation du projet

Objectif

3 Games Hub est une application web full-stack hébergeant trois mini-jeux indépendants accessibles depuis une interface commune. L'objectif est de proposer une expérience cohérente : authentification utilisateur, navigation entre les jeux, et classement global persisté en base de données.

Stack technique

Couche	Technologie
Frontend	HTML5 · CSS3 · JavaScript ES6 (vanilla, modules)
Backend	Node.js · Express.js
Base de données	MongoDB Atlas · Mongoose
Polices	Amanojaku, ChineseRock, Neon, Earthquake (OTF/TTF locales) + Space Grotesk (Google Fonts)
Sécurité	bcryptjs (hachage des mots de passe)

2. Architecture technique

Structure des dossiers

```
3-games-hub/  
├── backend/  
│   ├── index.js           # Serveur Express, point d'entrée  
│   ├── config/db.js      # Connexion MongoDB Atlas  
│   ├── models/User.js    # Schéma Mongoose utilisateur  
│   ├── controllers/  
│   │   └── authController.js  # Inscription / connexion
```

```

├── leaderboardController.js # Lecture et mise à jour des scores
├── routes/
│   ├── authRoutes.js      # POST /api/auth/signup, /login
│   └── leaderboardRoutes.js # GET /api/leaderboard, POST /score
└── frontend/
    ├── index.html          # Hub principal (carrousel)
    ├── jeux.html           # Catalogue des 3 jeux
    ├── classement.html     # Tableau de classement
    ├── css/
    │   ├── fonts.css       # @font-face locaux
    │   ├── style.css       # Hub principal
    │   ├── themes.css      # Surcharges par thème de jeu
    │   ├── auth.css        # Modal connexion / profil
    │   ├── jeux.css        # Page catalogue
    │   └── classement.css  # Page classement
    ├── js/
    │   ├── data.js         # Données statiques des 3 jeux
    │   ├── main.js         # Carrousel + lecture scores localStorage
    │   ├── auth.js         # Gestion session, modales login/profil
    │   ├── auth-guard.js   # Redirection si non connecté (jeux)
    │   └── classement.js   # Fetch leaderboard + rendu tableau
    └── games/
        ├── defi-de-lempereur/ # HTML + CSS + 5 fichiers JS
        ├── neon-pop/         # HTML + CSS + 8 fichiers JS
        └── sticky-climber/    # HTML + CSS + 5 fichiers JS

```

Séparation des responsabilités

Le backend Express sert les fichiers statiques du frontend (`express.static`) et expose l'API REST sous `/api/` . Ainsi, un seul serveur (port 5501) gère les deux préoccupations, ce qui évite les problèmes CORS en développement.

Côté frontend, chaque jeu est **isolé** dans son propre sous-dossier avec ses modules ES6. Le hub partage uniquement `auth.js` , `auth-guard.js` et `data.js` .

3. Conception des jeux

3.1 Le défi de l'Empereur — runner DOM à 3 voies

Principe : Le joueur se déplace entre 3 voies verticales et doit éviter des obstacles qui tombent du haut de l'écran.

Rendu : Entièrement en **DOM** (pas de canvas). Les obstacles sont des `<div>` créés dynamiquement et animés par CSS (`@keyframes tomberObstacle`), ce qui délègue le rendu au moteur CSS du navigateur.

Architecture de classes :

Classe	Rôle
Game	State machine (MENU → JEU EN COURS → GAME OVER), boucle <code>requestAnimationFrame</code> , spawn et collision
Joueur	Élément DOM <code>#joueurEl</code> , déplacement par <code>left = CENTRES_VOIES[voie] - 30px</code> , invincibilité avec animation <code>flash</code>
Obstacle	<code><div></code> appendé dans <code>#piste</code> , animation CSS <code>tomberObstacle</code> , auto-suppression sur <code>animationend</code>
initListeners	Clavier (← → / A D / Espace), touch (swipe), boutons menu

Gestion de la difficulté :

`vitesse = Math.max(800, 2200 - Math.floor(score / 10) × 100)` — la durée de chute décroît avec le score, rendant les obstacles de plus en plus rapides.

Détection de collision : Filtre préalable par voie (`o.voie !== this.joueur.voie`) puis test AABB via `getBoundingClientRect()`.

3.2 Neon Pop — réaction en chaîne (canvas)

Principe : Le joueur clique sur une bille pour déclencher une explosion qui propage une réaction en chaîne. Le score est proportionnel au nombre de billes touchées.

Architecture : 8 modules JS dont `Bille`, `Explosion`, `Particule`, `ObjetGraphique`. Le jeu est rendu sur un `<canvas>` HTML5 avec une boucle `requestAnimationFrame` (~582 lignes dans `game.js`).

3.3 Sticky Climber — grimpeur physique (canvas)

Principe : Le joueur lance un personnage (slime) qui s'accroche à des ancres pour grimper le plus haut possible. Le score est exprimé en mètres.

Rendu : Canvas full-screen (`window.innerWidth × window.innerHeight`), redessiné entièrement à chaque frame.

Système physique :

GRAVITE = 0.2 px/frame² (accélération verticale en chute libre)
VITESSE_ANGULAIRE = 0.07 rad/frame (rotation autour d'une ancre)
LONGUEUR_FIL_MAX = 130 px

À la release, la vitesse de lancement est calculée depuis la tangente du pendule :

```
vx = -sin(angle) × longueurFil × vitesseAngulaire  
vy = cos(angle) × longueurFil × vitesseAngulaire
```

Génération procédurale des ancrs :

4 niveaux de difficulté selon le score (< 30 m, < 70 m, < 130 m, ≥ 130 m). À chaque niveau, le rayon des ancrs diminue et les types `mobile` (oscillation sinusoïdale) et `éphémère` (durée de vie limitée) apparaissent.

Caméra : Un `offsetY` (négatif quand le joueur monte) scroll le monde. Le score est calculé par `Math.floor(-offsetY / 50)`. Le fond (ciel, montagne enneigée, nuages) utilise une parallaxe multi-couches à facteurs 0.02, 0.1.

4. Authentification et base de données

Modèle utilisateur (Mongoose)

```
UserSchema {  
  username   : String - unique, requis  
  password   : String - haché bcrypt (10 rounds)  
  avatar     : String - default "default-avatar.png"  
  gameScores : Map<String, Number> - clés : "emperor", "neon", "sticky"  
  globalScore: Number - somme des meilleurs scores par jeu  
}
```

Le type `Map` de Mongoose permet d'ajouter des jeux sans modifier le schéma. La mise à jour du score nécessite `user.markModified("gameScores")` avant `save()` pour forcer la détection du changement par Mongoose.

API REST

Méthode	Route	Fonction
POST	<code>/api/auth/signup</code>	Création de compte (validation + bcrypt)
POST	<code>/api/auth/login</code>	Connexion (comparaison bcrypt)
GET	<code>/api/leaderboard</code>	Liste des joueurs triés par <code>globalScore</code>
POST	<code>/api/leaderboard/score</code>	Mise à jour du meilleur score d'un jeu

Gestion de session

La session est stockée côté client dans `localStorage` (clé `game_hub_session` = nom

d'utilisateur). Ce choix, simple pour un projet académique, est complété par `auth-guard.js` qui redirige les visiteurs non connectés hors des pages de jeu.

Synchronisation des scores

À chaque fin de partie, chaque jeu envoie son **meilleur score localStorage** (pas seulement les nouveaux records) à `POST /api/leaderboard/score`. Côté backend, la mise à jour n'est effectuée que si le nouveau score dépasse la valeur enregistrée en base, garantissant l'idempotence.

5. Interface utilisateur et design

Système de thèmes CSS

Chaque jeu possède un thème (`theme-samurai` , `theme-neon` , `theme-sticky`) appliqué dynamiquement sur `<body>`. Le fichier `themes.css` surcharge les éléments communs (titre, boutons, polices) selon le thème actif :

Thème	Police titre	Couleur accentuée	Survol bouton
Samurai	ChineseRock	Fond rouge <code>#f35050</code>	Rouge
Neon	Neon Bright	Cyan <code>#00ffff</code>	Glow cyan
Sticky	Earthquake	Or <code>#ffd700</code>	Or

Pages

- **Hub** (`index.html`) : Carrousel entre les 3 jeux, fond thématique dynamique avec `backdrop-filter: blur`, score en localStorage affiché, navigation `← →`.
- **Catalogue** (`jeux.html`) : 3 cartes glassmorphism en grille, chaque carte avec l'image hero, la description et un bouton "Jouer". Grille responsive (3 → 2 → 1 colonne).
- **Classement** (`classement.html`) : Tableau unique avec 4 onglets (Global / par jeu). Les 3 premières places affichent les médailles   . La ligne du joueur connecté est mise en évidence.

6. Démarche avec l'assistance IA

Approche

Ce projet a été réalisé en **dialogue itératif** avec un assistant IA (Claude). L'IA n'a pas été laissée autonome : chaque fonctionnalité a été déclenchée par une instruction précise, et les

propositions ont été acceptées, rejetées ou redirigées au fil des échanges.

Comment les spécifications ont été formulées

Les instructions ont suivi un schéma récurrent :

1. **Contrainte technique explicite** — *"jeu runner à 3 voies uniquement en DOM, pas de canvas", "jeu de grapplin hook uniquement sur canvas"*.
2. **Objectif fonctionnel** — *"obstacles qui tombent, 3 vies, score qui monte", "s'accrocher et s'envoler avec un clic ou la barre espace"*.
3. **Contrainte de cohérence** — *"respecte les patterns de code de Neon Pop", "utilise les mêmes polices que le hub"*.

L'IA a ensuite proposé l'architecture de classes, les constantes physiques, et les structures CSS, tandis que les décisions visuelles et fonctionnelles finales restaient sous contrôle humain.

Exemples de redirections et corrections

Demande initiale IA	Correction apportée
Police AmanoJaku sur la page Jeux	<i>"Police neutre sur cette page"</i> → Space Grotesk
Fond sombre étoilé pour Sticky Climber	<i>"Fond clair avec montagne enneigée"</i> → refonte complète
Scores envoyés uniquement sur nouveau record	Diagnostic bug → envoi systématique du meilleur score localStorage
Classement déclenché manuellement	<i>"Vérifie que les scores sont bien sauvegardés"</i> → identification de <code>markModified</code> manquant

Bilan

L'IA a accéléré significativement l'implémentation (physique, génération procédurale, CSS avancé), mais la conception fonctionnelle, quoi construire, pour qui, avec quelles contraintes est restée entièrement à la charge du développeur